

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

Facultad de Ingeniería

Escuela Profesional de Ingeniería en Informática y Sistemas



PRÁCTICA N° 04:

“GESTIÓN DE HILOS”

Alumno	:	Mauricio José Flores Gómez
Código	:	2018-119061
Docente	:	Dr. Hugo Manuel Barraza Vizcarra
Curso	:	Sistemas Operativos
Ciclo	:	6to Ciclo
Año de estudios	:	3er año

Tacna - Perú

2026

I. OBJETIVOS

- Implementar la creación de hilos y gestión de hilos
- Analizar el rendimiento de un programa multihilo en comparación con un monohilo
- Investigar los beneficios y desafíos de la sincronización de hilos

II. FUNDAMENTO TEÓRICO

Hilo (Thread)

Un hilo (thread) es la unidad básica de ejecución dentro de un proceso. Los hilos permiten la ejecución concurrente de tareas, lo que mejora la eficiencia y la utilización de recursos en sistemas operativos

Beneficios de los hilos

- **Eficiencia en el uso de recursos:** Los hilos comparten el mismo espacio de direcciones de memoria, lo que reduce la sobrecarga de recursos en comparación con los procesos independientes.
- **Concurrencia:** Permiten que múltiples operaciones se realicen simultáneamente, mejorando el rendimiento en sistemas con múltiples núcleos.
- **Simplicidad:** Facilitan la implementación de tareas que pueden ejecutarse independientemente, como el manejo de eventos o la entrada/salida asíncrona.

Hilos a Nivel Kernel y a Nivel Usuario

- **Hilos a Nivel Kernel:**
 - Gestionados directamente por el sistema operativo.
 - Cada hilo tiene su propia entrada en la tabla de procesos del kernel.
 - Permiten una mejor administración de recursos del sistema.
- **Hilos a Nivel Usuario:**
 - Gestionados por bibliotecas de usuario.
 - El sistema operativo no es consciente de su existencia.
 - Su creación y destrucción es más rápida, pero limitan el uso de recursos del sistema.

Modelos Multihilos

- **Modelo Uno a Uno:** Cada hilo de usuario tiene un hilo correspondiente en el kernel, lo que permite una mayor paralelización.

- **Modelo Muchos a Uno:** Varios hilos de usuario están mapeados a un solo hilo de kernel, lo que puede ser más eficiente pero limita la paralelización.
- **Modelo Muchos a Muchos:** Varios hilos de usuario pueden ser mapeados a varios hilos de kernel, combinando los beneficios de ambos modelos.

Arquitecturas Multicore

Las arquitecturas multicore permiten que múltiples núcleos de procesamiento ejecuten hilos simultáneamente, lo que aumenta significativamente el rendimiento en aplicaciones paralelizadas. Este diseño es fundamental en la computación moderna.

Hilos POSIX

Los hilos POSIX (Portable Operating System Interface) proporcionan un estándar para la creación y gestión de hilos en sistemas Unix y Linux. La biblioteca pthread es una implementación de este estándar, ofreciendo funciones para crear, sincronizar y gestionar hilos, así como mecanismos de comunicación entre ellos. A continuación, se presentan algunas funciones clave de la biblioteca pthread, que permiten manejar hilos de manera efectiva:

- **pthread_create:** Crea un nuevo hilo.
- **pthread_join:** Espera a que un hilo termine su ejecución.
- **pthread_exit:** Termina la ejecución del hilo actual.
- **pthread_mutex_init:** Inicializa un mutex.
- **pthread_mutex_lock:** Bloquea un mutex.
- **pthread_mutex_unlock:** Desbloquea un mutex.
- **pthread_cond_wait:** Bloquea un hilo hasta que se recibe una señal en una variable de condición.
- **pthread_cond_signal:** Desbloquea un hilo que espera en una variable de condición.

III. PROCEDIMIENTO

3.1. Creación y gestión de hilos en C

Implementar la creación de dos hilos utilizando la biblioteca pthread.

Creamos el programa hilo1.c

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nano ...
~/Descargas/MSX88
GNU nano 8.7.1 hilo1.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* hilo_funcion(void* arg) {
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_exit(NULL);
}

int main() {
    pthread_t hilo1, hilo2;

    // Creación de los hilos
    pthread_create(&hilo1, NULL, hilo_funcion, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion, (void*)2);

    // Esperar a que los hilos terminen
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    printf("Finalizó la ejecución del programa principal\n");

    return 0;
}

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos el programa.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano hilo1.c
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ gcc -o hilo1 hilo1.c -lpthread
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./hilo1
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal
```

Registramos el tiempo de ejecución del programa utilizando el comando time.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ time ./hilo1
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal

real    0m0,037s
user    0m0,005s
sys     0m0,003s
```

3.2. Implementación de sincronización

Implementar un mecanismo de sincronización usando `pthread_mutex_t` para asegurar que solo un hilo acceda a una sección crítica del código

Creamos el programa hilo2.c

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nano ...
~/Descargas/MSX88
GNU nano 8.7.1 hilo2.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Declaración del mutex
pthread_mutex_t mutex;

// Función del hilo que utiliza el mutex
void* hilo_funcion_con_mutex(void* arg) {
    pthread_mutex_lock(&mutex); // Bloquear el mutex
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock(&mutex); // Desbloquear el mutex
    pthread_exit(NULL);
}

int main() {
    pthread_t hilo1, hilo2;

    // Inicializar el mutex
    pthread_mutex_init(&mutex, NULL);

    // Creación de los hilos
    pthread_create(&hilo1, NULL, hilo_funcion_con_mutex, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion_con_mutex, (void*)2);

    // Esperar a que los hilos terminen
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    // Destruir el mutex
    pthread_mutex_destroy(&mutex);

    printf("Finalizó la ejecución del programa principal\n");
    return 0;
}

^G Ayuda  ^O Guardar  ^F Buscar  ^K Cortar  ^T Ejecutar  ^C Ubicación
^X Salir  ^R Leer fich. ^\ Reemplazar ^U Pegar  ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos el programa.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano hilo2.c
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ gcc -o hilo2 hilo2.c -lpthread
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./hilo2
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal
```

Registramos el tiempo de ejecución del programa utilizando el comando time.

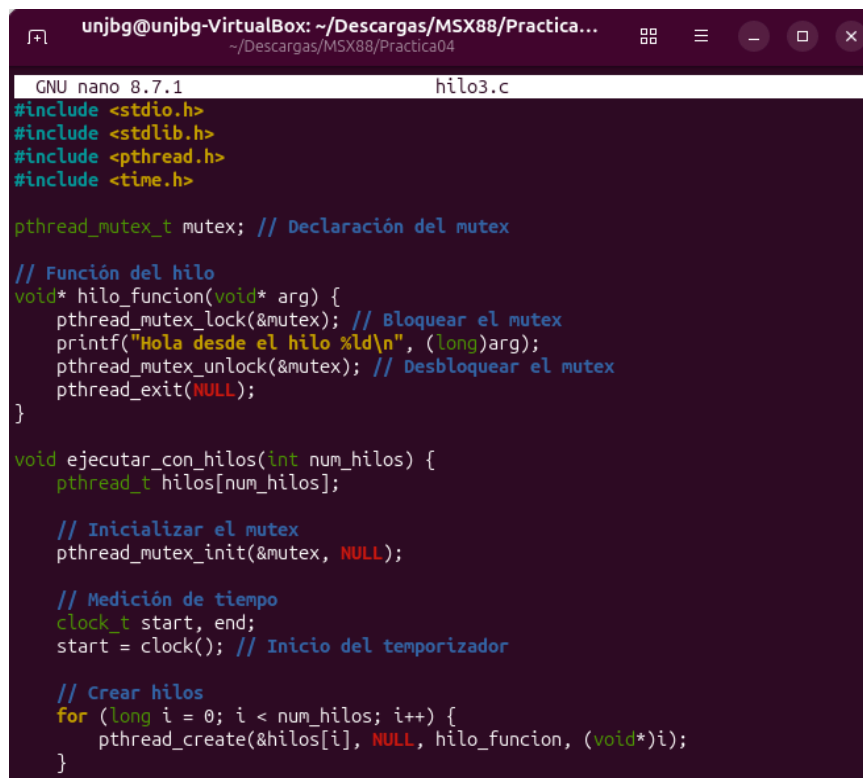
```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ time ./hilo2
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal

real    0m0,013s
user    0m0,006s
sys     0m0,000s
```

IV. ANÁLISIS DE RENDIMIENTO

Experimenta con el número de hilos y registra el tiempo de ejecución para diferentes configuraciones. Por ejemplo, intenta crear uno, cinco, diez hilos, etc. en lugar de dos y observa cómo afecta el rendimiento.

Creemos el programa hilo3.c



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88/Practica...
~/Descargas/MSX88/Practica04
GNU nano 8.7.1 hilo3.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

pthread_mutex_t mutex; // Declaración del mutex

// Función del hilo
void* hilo_funcion(void* arg) {
    pthread_mutex_lock(&mutex); // Bloquear el mutex
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_mutex_unlock(&mutex); // Desbloquear el mutex
    pthread_exit(NULL);
}

void ejecutar_con_hilos(int num_hilos) {
    pthread_t hilos[num_hilos];

    // Inicializar el mutex
    pthread_mutex_init(&mutex, NULL);

    // Medición de tiempo
    clock_t start, end;
    start = clock(); // Inicio del temporizador

    // Crear hilos
    for (long i = 0; i < num_hilos; i++) {
        pthread_create(&hilos[i], NULL, hilo_funcion, (void*)i);
    }
}
```

```

// Esperar a que los hilos terminen
for (int i = 0; i < num_hilos; i++) {
    pthread_join(hilos[i], NULL);
}

end = clock(); // Fin del temporizador

// Destruir el mutex
pthread_mutex_destroy(&mutex);

// Calcular y mostrar el tiempo total de ejecución
double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Tiempo total de ejecución con %d hilos: %f segundos\n",
       num_hilos, cpu_time_used);
}

int main() {
    printf("Ejecutando con 1 hilo...\n");
    ejecutar_con_hilos(1);

    printf("Ejecutando con 5 hilos...\n");
    ejecutar_con_hilos(5);

    printf("Ejecutando con 10 hilos...\n");
    ejecutar_con_hilos(10);

    printf("Finalizó la ejecución del programa principal\n");

    return 0;
}

```

^C Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
 ^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^/ Ir a línea

Compilamos y ejecutamos el programa.

```

unjbg@unjbg-VirtualBox:~/Descargas/MSX88/Practica04$ nano hilo3.c
unjbg@unjbg-VirtualBox:~/Descargas/MSX88/Practica04$ gcc -o hilo3 hilo3.c -lpthread
unjbg@unjbg-VirtualBox:~/Descargas/MSX88/Practica04$ ./hilo3
Ejecutando con 1 hilo...
Hola desde el hilo 0
Tiempo total de ejecución con 1 hilos: 0.000996 segundos
Ejecutando con 5 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Tiempo total de ejecución con 5 hilos: 0.000207 segundos
Ejecutando con 10 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Hola desde el hilo 5
Hola desde el hilo 6
Hola desde el hilo 7
Hola desde el hilo 8
Hola desde el hilo 9
Tiempo total de ejecución con 10 hilos: 0.000307 segundos
Finalizó la ejecución del programa principal

```

Registramos el tiempo de ejecución del programa utilizando el comando time.

```

unjbg@unjbg-VirtualBox:~/Descargas/MSX88/Practica04$ time ./hilo3
Ejecutando con 1 hilo...
Hola desde el hilo 0
Tiempo total de ejecución con 1 hilos: 0.000970 segundos
Ejecutando con 5 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 4
Hola desde el hilo 2
Hola desde el hilo 3
Tiempo total de ejecución con 5 hilos: 0.000508 segundos
Ejecutando con 10 hilos...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 3
Hola desde el hilo 4
Hola desde el hilo 5
Hola desde el hilo 6
Hola desde el hilo 7
Hola desde el hilo 8
Hola desde el hilo 9
Tiempo total de ejecución con 10 hilos: 0.000432 segundos
Finalizó la ejecución del programa principal

real    0m0,013s
user    0m0,005s
sys     0m0,001s

```

Observe como varía el tiempo de ejecución al aumentar el número de hilos.

Se comprobó que el uso de múltiples hilos reduce considerablemente el tiempo de ejecución en comparación con un entorno monohilo, pasando de 0.000996 s con 1 hilo a 0.000207 s con 5 hilos. Sin embargo, al aumentar a 10 hilos el tiempo subió a 0.000307 s, lo que demuestra que existe un límite donde el rendimiento deja de mejorar debido a la sobrecarga por cambio de contexto en el procesador.

V. ANÁLISIS DE RESULTADOS

Para esta práctica, deberá elaborar un documento donde se respondan las siguientes cuestiones.

1. Al comparar el rendimiento de un programa multihilo con un monohilo, **¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?**

La diferencia fue que el tiempo de CPU bajó de 0.000996 s a 0.000207 s al pasar de 1 a 5 hilos, demostrando que la concurrencia optimiza la ejecución. Una buena gestión de hilos aprovecha los núcleos de la CPU sin sobrecargar el sistema.

2. **¿Qué funciones de la biblioteca pthread utilizaste en tu implementación?** Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.

Se usaron `pthread_create` (para iniciar cada hilo), `pthread_join` (para esperar a que terminen antes de cerrar el programa) y `pthread_mutex_lock` (para proteger la sección crítica y evitar interferencias).

3. **¿Cómo se comportó el programa al aumentar el número de hilos más allá de un cierto punto? ¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó?** Explica por qué esto puede ocurrir en un entorno multihilo.

Al subir a 10 hilos el tiempo aumentó a 0.000307 s. Esto pasa porque crear demasiados hilos genera sobrecarga por cambio de contexto en la CPU y esperas acumuladas por el uso de mutex

4. Durante la ejecución, **¿observaste alguna condición de carrera o comportamiento inesperado?** Si es así, **¿cómo impactó esto en los resultados obtenidos y qué medidas se podrían tomar para prevenirlo?**

Las impresiones en consola salieron desordenadas debido al planificador del sistema. Para evitar condiciones de carrera y la corrupción de datos compartidos se deben usar mecanismos de sincronización como `pthread_mutex_t`.

VI. CONCLUSIONES

- Se implementó correctamente la biblioteca `pthread` en C, comprobando que la creación y gestión de hilos mediante `pthread_create` y `pthread_join` permite ejecutar tareas concurrentes dentro de un mismo proceso de forma estructurada.
- El análisis de rendimiento demostró que la ejecución multihilo reduce significativamente los tiempos en comparación con un entorno monohilo; no obstante, exceder la capacidad del procesador provoca sobrecarga por cambio de contexto y disminuye la eficiencia.
- Se verificó que el uso de mecanismos de exclusión mutua (`pthread_mutex`) es indispensable para prevenir condiciones de carrera en secciones críticas, garantizando el orden y la integridad de los datos en aplicaciones concurrentes.